

Search implementation and measurement

Scope

Implemented questions 1-7 in `search.py` and `searchAgents.py` only. Existing interfaces, framework code, expansion counters, and Berkeley attribution are retained. The ZIP contains the full source package and layouts. `performance.csv` contains the measured data.

Environment and timing

CPython 3.10.10 on Windows 10.0.26200. Python 3.11, recommended by the assignment, is not installed and was not tested. Times are medians of three searches using `time.perf_counter` with fresh problems and cold heuristic caches. Garbage collection runs before each measurement. Timing excludes imports, layout loading, problem construction, graphics, and plan execution.

Memory and nodes

Peak memory is measured in a separate fresh run using `tracemalloc`, including search allocations, bookkeeping, and heuristic caches. This is traced Python allocation, not process RAM; pre-existing maze and interpreter memory are excluded. One KiB is 1024 bytes. Nodes are `problem._expanded` before plan replay. Heuristic distance-map BFS work is included in time and memory but not the node counter.

Algorithms

DFS uses `util.Stack` and marks states on removal for expansion, preserving successor order. BFS uses `util.Queue` and marks states on enqueue. UCS uses A* with the zero heuristic. A* uses `util.PriorityQueue` with `g+h` priority, tracks cheapest known path costs, skips stale entries, and reopens states when a cheaper route is found. Exhausted searches return an empty action list.

State and heuristics

Corners states contain position and immutable visited-corner flags. The corner heuristic minimizes Manhattan tour length over remaining corner orders, ignoring walls. The food heuristic adds nearest-food maze distance to a minimum spanning tree over remaining food using exact maze distances. BFS distance maps and tree costs are cached on the problem.

Tradeoffs

Fringe entries store action lists, so extending a route copies its prefix. Parent links could reduce this memory cost but add complexity. Heuristic caches trade memory and preprocessing for fewer search expansions. Small timing differences are machine-dependent; fewer nodes do not necessarily imply lower runtime.

Fixed-position search results

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyMaze	DFS	10	15	0.074	3.9
tinyMaze	BFS	8	15	0.071	3.9
tinyMaze	UCS	8	15	0.077	4.2
tinyMaze	A* Manhattan	8	14	0.083	4.2
mediumMaze	DFS	130	146	0.581	30.9
mediumMaze	BFS	68	269	0.974	39.0
mediumMaze	UCS	68	269	1.106	41.1
mediumMaze	A* Manhattan	68	221	0.997	38.5
bigMaze	DFS	210	390	1.581	116.8
bigMaze	BFS	210	620	2.452	103.3
bigMaze	UCS	210	620	2.657	90.1
bigMaze	A* Manhattan	210	549	2.714	88.8
openMaze	DFS	298	576	3.229	562.5
openMaze	BFS	54	682	2.642	99.5
openMaze	UCS	54	682	3.106	123.1
openMaze	A* Manhattan	54	535	2.466	85.6

Solution quality

BFS, UCS, and A* agree on shortest path length in all four unit-cost mazes. DFS takes 130 steps on mediumMaze versus 68 for BFS, and 298 on openMaze versus 54. Its optimal bigMaze route is incidental. The supplied mediumMaze differs from the approximate DFS length in the PDF; successor order is preserved and returned moves were checked.

Search effort

A* expands 549 bigMaze nodes versus UCS at 620, matching the reference counts. Runtime is slightly higher: 2.714 versus 2.657 ms. On openMaze, A* reduces nodes from 682 to 535 and time from 3.106 to 2.466 ms. Heuristic overhead matters at this scale.

Memory and movement

DFS uses 562.5 KiB on openMaze versus 99.5 KiB for BFS with stored action lists, so DFS is not invariably cheaper in memory. Expanded states describe planning; Pac-Man follows only the returned plan and need not visit every examined square. BFS minimizes steps, while UCS handles weighted cost.

Corners, food, and weighted search

Maze	Strategy	Steps	Nodes	Time (ms)	Peak (KiB)
tinyCorners	BFS	28	252	1.419	85.1
tinyCorners	A* corners	28	159	3.009	84.3
mediumCorners	BFS	106	1966	11.189	678.3
mediumCorners	A* corners	106	741	19.303	317.9
testSearch	UCS	7	14	0.419	16.8
testSearch	A* food MST	7	7	0.285	13.1
tinySearch	UCS	27	5057	219.103	8400.2
tinySearch	A* food MST	27	89	5.945	264.4
trickySearch	UCS	60	16688	1485.511	48296.3
trickySearch	A* food MST	60	255	30.424	1065.6
mediumDottedMaze	UCS east	74	186	0.987	41.1
mediumScaryMaze	UCS west	92	108	0.609	30.1

Corners

Both methods find 28 steps on tinyCorners and 106 on mediumCorners. A* reduces mediumCorners nodes from 1966 to 741, below the 1200-node full-credit threshold and 800-node stretch marker. Peak memory falls from 678.3 to 317.9 KiB. Runtime rises from 11.189 to 19.303 ms because heuristic evaluation outweighs saved expansions.

Food

Food A* agrees with UCS on costs 7, 27, and 60. On trickySearch, nodes fall from 16688 to 255, below the 7000-node extra-credit threshold. Runtime falls from 1485.511 to 30.424 ms and peak memory from 48296.3 to 1065.6 KiB.

Weighted costs

The table reports steps, not weighted costs. StayEast UCS takes 74 steps with cost 1.000976583804004. StayWest UCS takes 92 steps with cost 68719479864. These low and high costs reflect the supplied exponential cost functions. Both agents won their headless games.

Correctness and critical assessment

Validation

All 28 benchmark cases passed legal-action replay and reached the problem goal. Targeted checks passed for start-at-goal, unreachable goals, cycles, cheaper duplicate routes, and A* reopening with an admissible inconsistent heuristic. BFS/UCS/A* solved a supplied eight-puzzle instance; its unchanged wrapper uses a module global, initialized by the check as its main program does.

Heuristic checks

Exhaustive consistency checks passed 912 transitions across 384 reachable tinyCorners states and 17732 transitions across 8100 tinySearch states. Values were nonnegative and zero at goals. A* and UCS agreed on solution lengths. A separate check verified starting-corner visitation and that successor generation does not mutate parent states.

Why the bounds are valid

Removing walls relaxes corner navigation: the shortest Manhattan tour is a lower bound, and a legal step followed by a relaxed tour proves consistency. Any food completion must reach a dot and then connect all food, so nearest-food distance plus the food MST is a lower bound. Without eating, nearest distance changes by at most one. When eating, the removed dot can be attached to an MST of remaining dots at its nearest-food distance, proving consistency. Both bounds vanish at goals.

Limits

Source compilation and representative headless games passed. No autograder or test_cases directory was supplied; these results are not an official grade. Python 3.11 was unavailable. mediumSearch was not benchmarked. Timing and memory measurements are specific to this environment. Existing educational-use licensing and attribution remain intact.

Critical assessment

BFS is a useful baseline for unit-cost shortest paths; UCS handles varying nonnegative costs. DFS may find a route with fewer expansions but offers no shortest-path guarantee. A* helps most when guidance offsets heuristic evaluation cost. The food MST delivers a large improvement in all three measured food mazes. The corner heuristic meets the node target and reduces memory but is slower than BFS here. Compare solution cost, expansions, runtime, and memory together instead of ranking algorithms by node count alone.